

Génération de Labyrinthes Pac-Man

UE Travaux d'Études et de Recherche – Semestre 2

Ikram Benchalal Nada Zina Aya Haddoun

Université Côte d'Azur
Master 1 Informatique – Parcours IA

Superviseur : Gilles Menez

13 mars 2026

Sommaire

- 1 [Algorithmes de génération](#)
- 2 [Web Service & API](#)
- 3 [Tests](#)
- 4 [CI/CD](#)
- 5 [Interface graphique](#)
- 6 [Projet & Collaboration](#)
- 7 [Références](#)

Vue d'ensemble : DFS vs Prim

DFS



Creuse en profondeur
Longs couloirs sinueux

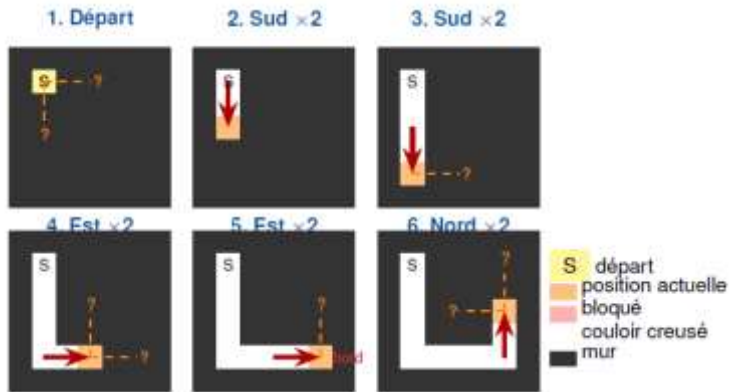
Prim



Choisit un mur au hasard
Structure étalée, ouverte

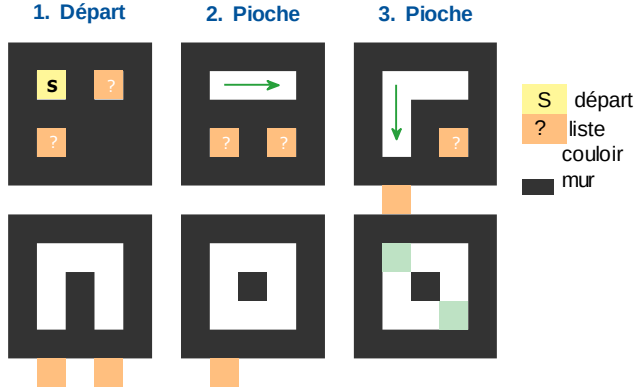
→ Les deux génèrent des labyrinthes **parfaits** (un seul chemin entre chaque paire de cases)

DFS pas à pas - Évolution étape par étape



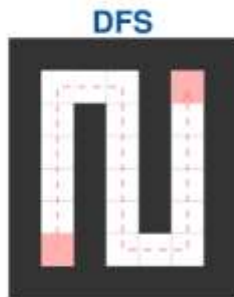
→ Règle : tant qu'on peut avancer → on avance | Sinon → backtrack

Prim pas à pas – Évolution étape par étape



✂ Pioche AU HASARD dans la **liste** $\neq$ DFS qui suit le dernier point

Benchmark : DFS vs Prim



✗ 2 culs-de-sac
✗ 0 intersection

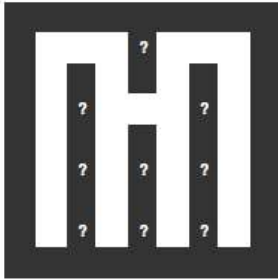


✓ 0 cul-de-sac
✓ 5 intersections

→ **Prim**
retenu

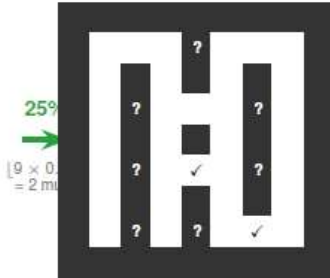
Ajout de boucles - add_loops(maze, loop_percent)

0% boucles (parfait)



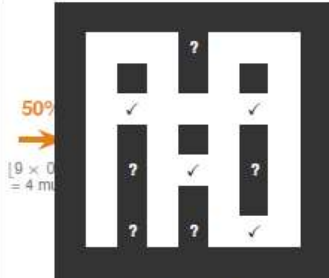
9 murs candidats
1 seul chemin entre 2 cases
sur mur = mur candidat

25% boucles (Pac-Man)



2 boucles créées
choix de chemins → jouable
✓ = mur supprimé (boucle)

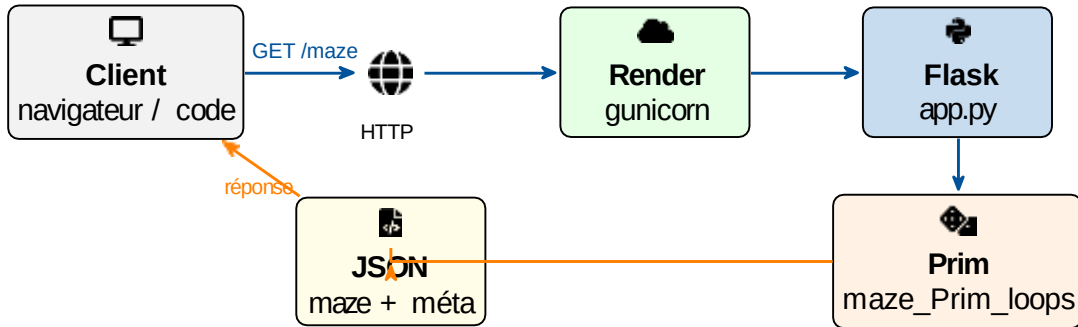
50% boucles (trop ouvert)



4 boucles — trop de raccourcis
le labyrinthe perd son intérêt
noir = mur | blanc = couloir

Pac-Man $\approx$ 25% | Trop facile $>$ 50% | `add_loops(maze, %)` supprime un % des murs candidats

Architecture globale



URL : https://ter-s2-d.onrender.com/maze?width=21&height=21&loop_percent=25

API : Routes & Réponse JSON

Routes

GET / Health check
GET /maze Générer un maze

Paramètres de /maze :

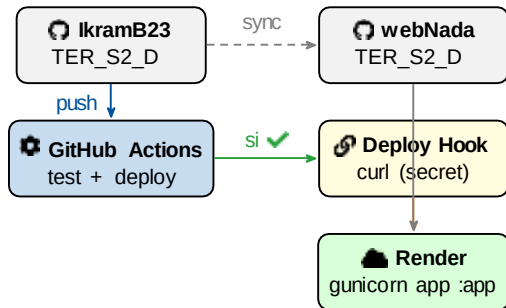
width int défaut : 21
height int défaut : 21
loop_percent int défaut : 25

Réponse JSON



Convention : 0 = mur, 1 = couloir

Déploiement Render & Deploy Hook



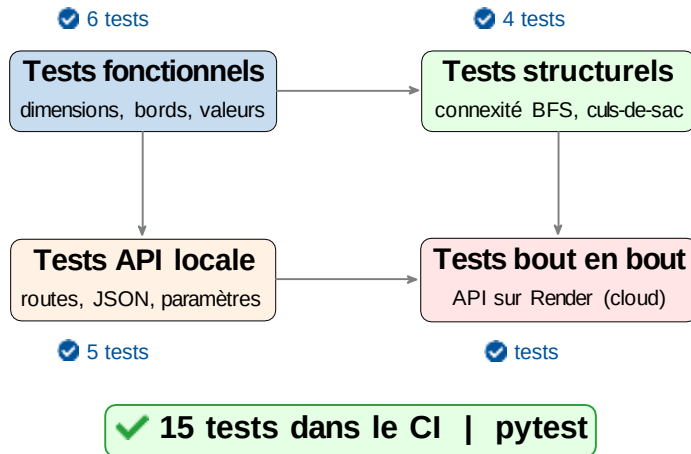
Stack technique

-  Python / Flask
-  Gunicorn (prod)
-  Render (gratuit)
-  2 repos synchro
-  Secret GitHub

Note

Le serveur Render “dort” après 15 min
Premier appel ~30s de réveil

Stratégie de tests : 4 niveaux



Exemples de tests clés

Test fonctionnel : dimensions

```
def test_dimensions ():  
    maze = generate_pacman_maze (21, 21)  
    assert len(maze) == 21  
    assert len(maze[0]) == 21
```

Test structural : connexité BFS

```
def test_connectivity ():  
    # BFS depuis un couloir  
    # verifie qu'on atteint TOUS  
    # les couloirs du labyrinthe  
    visited = bfs(maze, start)  
    assert len(visited) == total
```

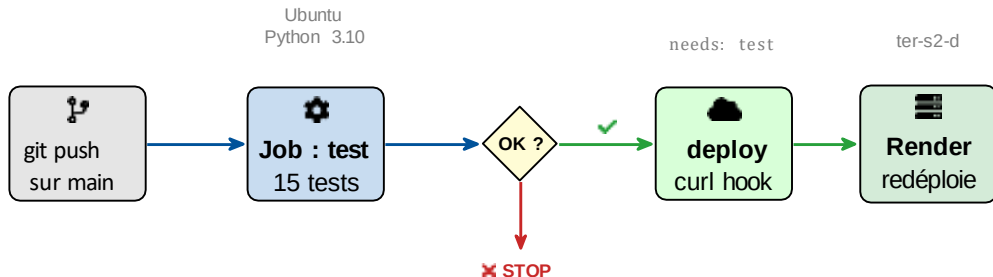
Test API locale

```
def test_maze_route ():  
    client = app.test_client()  
    resp = client.get('/ maze')  
    assert resp.status_code == 200  
    data = resp.get_json()  
    assert data["width"] == 21
```

Test bout en bout

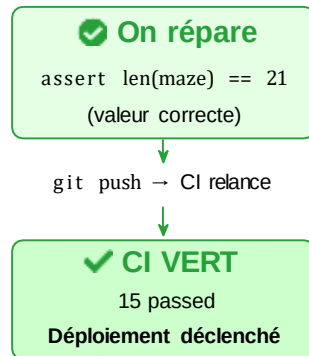
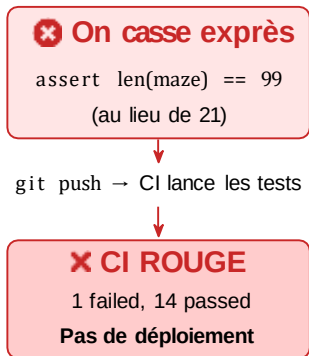
```
def test_cloud ():  
    URL = "https://ter-s2-d.  
          onrender.com/maze"  
    resp = requests.get(URL)  
    assert resp.status_code == 200
```

Pipeline CI/CD – GitHub Actions



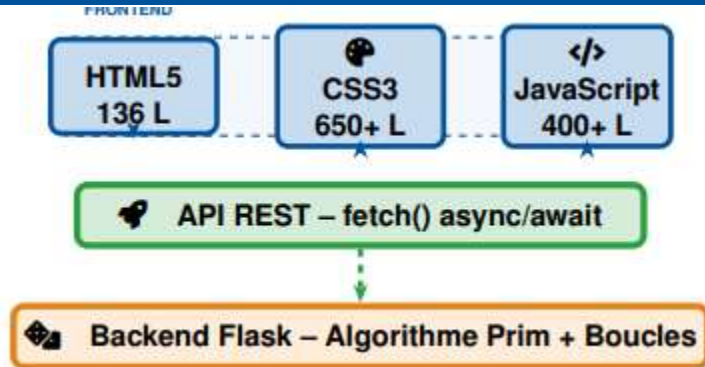
 [.github/workflows/ci.yml](#) - déclenché automatiquement à chaque push

Démo rouge / vert



Le CI **protège main** : code cassé ≠ déployé en production

Interface graphique – Architecture



HTML5	CSS3	JS + Fetch
Formulaires Canvas 500×500 Stats dynamiques	Responsive design 2 col desktop Animations fluides	Validation Appel async Rendu canvas 2D

Implémentation technique

HTML5

- Input width 5-101
- Input height 5-101
- Slider boucles 0-100%
- Canvas 500×500
- Stats dynamiques

</> JavaScript

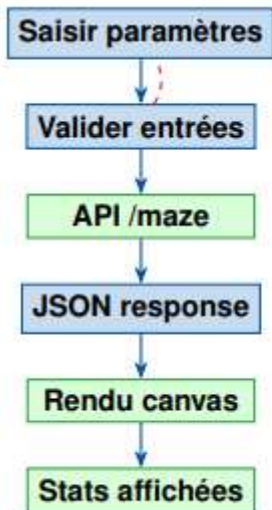
- 1 Validation inputs
- 2 Appel API async
- 3 Rendu canvas 2D
- 4 Calcul stats
- 5 Export PNG/JSON

CSS3

- 2 colonnes desktop
- 1 colonne mobile
- Gradients modernes
- Animations fluides

Gestion erreurs

- try/catch fetch
- Messages utilisateur
- Status HTTP



Fonctionnalités



Responsive

- Desktop 2 col
- Tablet adapté
- Mobile lisible



Export

- PNG image
- JSON données



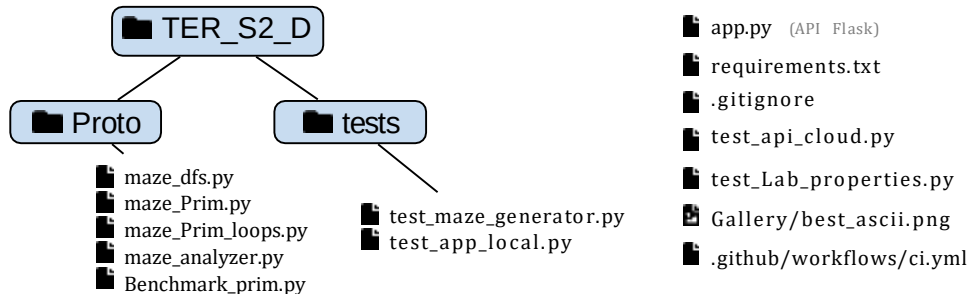
UX

- Temps réel
- Animations

Résultat Interface



Structure du projet



Choix techniques

Prim (pas DFS)	plus adapté Pac-Man
Fonctions (pas classes)	plus testable
Flask (pas Django)	léger et simple
Render (gratuit)	déploiement facile
Dimensions impaires	structure de grille

Répartition & Collaboration



Ikram

- Algo DFS + adaptation Prim
- Benchmark comparatif
- Tests (pytest)
- CI/CD (GitHub Actions)
- Gallery + .gitignore + sync repos



Nada

- Algo Prim (original, classe) , Benchmark
- API Flask + app.py
- Déploiement Render
- Tests cloud (end-to-end)



Aya

- Interface graphique (GUI)
- Appel API + affichage visuel
- Tests (pytest) + CI/CD



GitHub

Repo principal : IkramB23

Fork Nada : webNada

Branches : main, ikram-work, nada_work,
aya-Work

Merge sur main

Prof : gmenez (collaborateur)

Références

-  Render – Your First Deploy <https://render.com/docs/your-first-deploy>
-  Flask – Documentation officielle <https://flask.palletsprojects.com/>
-  Real Python – API Integration <https://realpython.com/api-integration-in-python/>
-  pytest – Documentation <https://docs.pytest.org/>
-  GitHub Actions – Documentation <https://docs.github.com/en/actions>
-  Algorithmique 2 – Cours de M.Christophe Crespelle <https://lms.univ-cotedazur.fr/2024/course/view.php?id=7049§ion=3#tabs-tree-start>
-  Algorithme de Prim – Wikipédia https://fr.wikipedia.org/wiki/Algorithme_de_Prim
-  DFS Maze generation https://en.wikipedia.org/wiki/Maze_generation_algorithm
-  Google Gemini <https://gemini.google.com/app?hl=fr>

Merci !

Questions ?

 github.com/lkramB23/TER_S2_D

 ter-s2-d.onrender.com